

Week 6, Lecture 2 WOD: Answer Key

Find the Bugs

NRES 746, Instructor Answer Key (not for student distribution)

Each item repeats the handout's question, then gives the answer, a short rationale, and mistakes to watch for. Both buggy functions are run below, so the symptoms described are real R output. The corrected `detect_prob()` matches the Lab 4, Act 2 key and is built live in today's demo. `sim_detect()` is a stand-alone stand-in with the same loop pattern as the demo's `estimate_power()`.

Part 1: Find and fix

1a.

`detect_prob()`: circle the bug, write the corrected line, and say what would go wrong if you ran the buggy version.

Answer. `1 - pobs^k` should be `1 - (1 - pobs)^k`.

Why. An animal is missed on one person-day with probability $1 - p_{obs}$, and missed on all k days with probability $(1 - p_{obs})^k$. The buggy line uses the chance of being *detected* every day instead. The result is a detection probability near 1, so the simulated counts would be close to the true abundance:

```
detect_prob_bug <- function(k, pobs = 0.02) 1 - pobs^k
c(buggy = detect_prob_bug(3), fixed = detect_prob(3))
```

```
##      buggy      fixed
## 0.999992 0.058808
```

Watch for. Writing `(1 - pobs)^k` without the leading `1 -`. That is the probability of being missed, not detected.

1b.

`sim_detect()`: circle the bug, write the corrected line, and say what would go wrong if you ran the buggy version.

Answer. `detected <- n_det > 0` should be `detected[i] <- n_det > 0`.

Why. Without the index, each pass through the loop overwrites the whole vector with a single `TRUE` or `FALSE`. After the loop, `detected` holds only the last survey, so the result is always exactly 0 or 1 instead of about $1 - 0.6^3 = 0.784$:

```
sim_detect_bug <- function(R = 1000, visits = 3, p = 0.4) {
  detected <- logical(R)
  for (i in seq_len(R)) {
    n_det <- rbinom(1, size = visits, prob = p)
    detected <- n_det > 0
  }
  mean(detected)
}
sim_detect <- function(R = 1000, visits = 3, p = 0.4) {
  detected <- logical(R)
```

```

for (i in seq_len(R)) {
  n_det <- rbinom(1, size = visits, prob = p)
  detected[i] <- n_det > 0
}
mean(detected)
}
set.seed(3)
replicate(8, sim_detect_bug())

```

```
## [1] 1 1 1 1 0 1 1 1
```

```
c(fixed = sim_detect(), exact = 1 - 0.6^3)
```

```
## fixed exact
```

```
## 0.758 0.784
```

Watch for. Students who fix it by moving `mean()` inside the loop. That still keeps only one replicate's result.

Part 2: Catch it next time

For one of the bugs, what quick check (one line of code and the answer you'd expect) would have caught it?

Sample answers (either is fine).

- **1a:** `detect_prob(1)` should return 0.02 (one day of effort should equal `pobs`).
- **1b:** `sim_detect()` should come out near $1 - (1 - 0.4)^3 = 0.784$, the same formula as 1a. At minimum, it should never be exactly 0 or 1.

Why. Good checks use cases where you already know the answer: an edge case ($k = 1$) or an exact answer the simulation should approximate. Students who notice that 1a's formula gives 1b's exact answer have made a nice connection: a simulation can be checked against math whenever the math is available. These are the same kinds of checks to demand from AI-written code in today's demo.

Watch for. "Run it and see if the answer looks reasonable." Bug 1a gives counts that look plausible unless you know what to expect. A check needs a specific expected value.